

Monolithic Architecture

Monolithic Architecture: When to Use, Benefits and Challenges

Definizione

L'architettura monolitica è un approccio tradizionale in cui l'intera applicazione viene progettata, sviluppata e distribuita come un'unica unità unificata.

Tutte le funzionalità — interfaccia utente, endpoint API, logica di business, servizi, accesso ai dati — sono contenute in **un'unica codebase** e distribuite come **un unico blocco**.

Punto fondamentale spesso frainteso: anche se un'applicazione monolitica è **ben progettata internamente con livelli separati** (presentation layer, business logic layer, data access layer), resta monolitica, perché *tutte queste componenti vengono comunque impacchettate e distribuite come un'unica entità*.

Esempi di artefatto di deploy:

- nel mondo Java: **un singolo file JAR o WAR**
- nel mondo .NET/web: **una singola applicazione web**

Quindi la formula è: **unica codebase + unico grande deployment**.

Osservazione importante della lezione: la maggior parte delle applicazioni legacy sono monolitiche, **ma questo non rende l'architettura monolitica obsoleta o arretrata**. In molte situazioni è ancora *l'opzione migliore e più realistica*.

Quando usare l'architettura monolitica (i 5 scenari)

Scenario	Motivazione
Startup con team piccolo (2–5 persone)	La semplicità permette di concentrarsi sul prodotto e soddisfare rapidamente i requisiti, senza l'overhead di coordinamento e le complessità di un sistema distribuito .
Logica di business semplice, scalabilità prevedibile	Se non prevedi requisiti di scala massicci e imprevedibili, il monolite è ideale e molto più facile da gestire.
Validazione rapida di un'idea (MVP / time-to-market)	Permette iterazioni e sviluppo veloci. Validi l'idea di business senza investire eccessivamente in infrastrutture complesse .
Team senza esperienza di sistemi distribuiti	<i>"Siamo onesti, i microservizi presentano notevoli complessità."</i> Meglio partire dal monolite, sfruttare le competenze esistenti e minimizzare il rischio di errori o ritardi costosi .
Serve forte coerenza (strong consistency)	Un monolite garantisce facilmente coerenza forte all'interno di un singolo processo e di un singolo database (transazioni ACID locali).

Per l'applicazione del corso — target dichiarato: **2.000 utenti simultanei e 500 richieste al secondo** — il monolite è un punto di partenza molto ragionevole, anche prevedendo crescita futura. La strategia è **"start simple, scale later"**.

I 6 vantaggi (Benefits)

1. **Facile da sviluppare** — è l'approccio universalmente compreso; quasi ogni team ha già competenze e strumenti.
2. **Debug più semplice** — con una sola codebase puoi tracciare il codice direttamente all'interno dello stesso processo, senza saltare tra servizi.
3. **Deployment meno complesso** — gestisci **un singolo artefatto** e distribuisce quello.
4. **Scaling semplice per carichi moderati** — spesso basta **allocare più risorse al server** che esegue il monolite (scaling verticale).
5. **Meno componenti da monitorare, gestire e proteggere** rispetto a un sistema distribuito.
6. **Nessuna latenza di rete tra i moduli** — tutti i moduli girano nello stesso processo, quindi le chiamate tra loro sono **chiamate in-process** (nanosecondi), non chiamate HTTP (millisecondi).

Le 5 sfide (Challenges)

1. **Complessità crescente** → "**Big Ball of Mud**"
2. Aggiungendo funzionalità, la codebase diventa enorme e complessa. Diventa difficile per chiunque — **soprattutto per i nuovi membri del team** — comprenderla e gestirla.
3. **Modifiche rischiose e lente**
4. In un codice grande e strettamente accoppiato, gli sviluppatori diventano **restii a modificare**, perché una piccola modifica in un'area può avere conseguenze indesiderate altrove. Questo rallenta il rilascio di nuove funzionalità.
5. **Deployment monolitico**
6. *Anche una piccola modifica a un singolo modulo richiede la ricostruzione e il redeploy dell'intera applicazione.* Risultato: rilasci lenti, rischiosi e **meno frequenti**.
7. **Lock-in tecnologico**
8. È molto difficile adottare una nuova tecnologia, framework o linguaggio **solo per una parte** dell'applicazione. Sei vincolato a uno stack unico per l'intero sistema. Aggiornare significa spesso riscrivere ampie porzioni.
9. **Scalabilità non granulare**
10. Se un **singolo modulo** è sotto carico elevato, devi comunque **scalare l'intera applicazione**, comprese tutte le parti che non ne hanno bisogno. È **inefficiente e costoso**. Non puoi scalare i componenti in modo indipendente.

Esempio concreto: nel Black Friday è il modulo *Catalog* (navigazione prodotti) a ricevere il 90% del traffico, mentre *Ordering* riceve molto meno. In un monolite devi replicare tutto — catalogo, ordini, pagamenti, account — su ogni nuova istanza, pagando risorse per moduli che sono inattivi.

Foundational Design Principles: KISS, YAGNI, DRY

Il problema che risolvono

"Un errore comune, soprattutto per noi ingegneri che amiamo risolvere problemi, è quello di progettare sistemi più complessi di quanto i requisiti effettivi giustifichino."

Questi principi agiscono come **bussola**, guidando verso progetti più puliti, efficienti e **appropriati**.

KISS — "Keep It Simple, Stupid" (o "Keep It Simple and Straightforward")

Definizione: valorizzare la semplicità nei progetti. La maggior parte dei sistemi **funziona meglio quando è il più semplice possibile**, piuttosto che eccessivamente complessa. La semplicità deve essere un **obiettivo primario**, e ogni complessità non necessaria va **attivamente evitata**.

Come si applica concretamente:

- usare **nomi significativi** per componenti, API e metodi, che riflettano accuratamente le loro responsabilità
- garantire una **chiara separation of concerns** tra livelli e moduli
- **rimuovere completamente** funzionalità o logica non necessarie ai requisiti attuali

Perché conviene: un sistema semplice è più facile da comprendere, mantenere, debuggare e far evolvere.

YAGNI — "You Aren't Gonna Need It"

Definizione: non costruire funzionalità o aggiungere complessità **per requisiti che non esistono ancora** e che non sono chiaramente definiti al momento.

Affronta direttamente la tendenza di sviluppatori e architetti a **pensare troppo in là nel futuro**, aggiungendo funzionalità, componenti o livelli di astrazione extra "nel caso servissero". Deriva dall'Extreme Programming.

Esempio: aggiungere oggi un message broker "perché un domani forse ci servirà la comunicazione asincrona" — quando hai 200 utenti e nessun caso d'uso asincrono — è una violazione di YAGNI. Paghi complessità, infrastruttura e manutenzione **oggi** per un beneficio **ipotetico**.

DRY — "Don't Repeat Yourself"

Definizione (dal libro *The Pragmatic Programmer*): **ogni informazione deve avere una rappresentazione unica, univoca e affidabile all'interno del sistema**.

Se una logica, una configurazione o un elemento di progettazione compare in più punti, stai violando DRY.

Perché la duplicazione è dannosa — due motivi espliciti nella lezione:

1. Se devi modificare quella logica, **devi ricordarti di modificarla in ogni punto duplicato** (e prima o poi te ne dimentichi uno → bug)
2. La duplicazione rende codice e progettazione **più grandi e più difficili da comprendere**

Come guidano il design dell'e-commerce

Le tre domande che il corso si porrà costantemente:

Principio	Domanda da porsi
KISS	"È questo il modo più semplice per ottenere questa funzionalità? C'è complessità superflua da eliminare?"
YAGNI	"Questo è un requisito concreto e attuale? Se no, non lo implemento."
DRY	"Sto duplicando logica o funzionalità tra unità concettuali diverse?"

L'obiettivo dichiarato: un'architettura iniziale **pulita, focalizzata, facile da comprendere e facile da far evolvere**.

Design: E-Shop Monolithic App w/ KISS & YAGNI

Qui si entra nel passo **DESIGN** del ciclo.

Il "Design Toolkit" (metodo dell'architetto)

Punto metodologico importante: *prima di disegnare qualsiasi progetto architeturale, un buon architetto rivede gli strumenti di progettazione disponibili per il problema in questione*. Il "toolkit" ha 4 caselle che verranno riempite ad ogni iterazione:

1. Requisiti funzionali (invariati dalla Lezione 1)

- navigare i prodotti
- aggiungere prodotti al carrello
- controllare il carrello
- effettuare un ordine e vedere gli ordini precedenti

2. Requisiti non funzionali (per questa fase iniziale)

Requisito	Target dichiarato
Disponibilità	Alta disponibilità, 24/7
Performance / concorrenza	Fino a 2.000 utenti simultanei — <i>"non puntiamo ancora a milioni"</i>
Tempo di risposta	Caricamento pagina in 2–3 secondi
Semplicità	Facile da sviluppare, distribuire e mantenere

3. Vincoli e assunzioni

- test di mercato iniziale rapido
- **team di sviluppo piccolo**
- **budget limitato** → evitare infrastrutture complesse o costose
- il concetto di business **necessita di validazione** → funzionalità e requisiti potrebbero evolvere

4. Conoscenze architetturelle acquisite

- Pattern: **architettura monolitica**
- Principi: **KISS, YAGNI, DRY**

Il toolkit **cresce a ogni lezione**: è il "magazzino" da cui l'architetto pesca la soluzione. Per ora contiene poco, ed è esattamente ciò che serve.

Il design risultante

Il diagramma è: **Client** → **una singola applicazione (application server)**. Letteralmente **una sola scatola**.

La lezione anticipa l'obiezione dello studente — *"abbiamo studiato tutto questo per arrivare a una sola scatola?"* — e risponde **sì**, perché il design riflette direttamente l'applicazione dei principi:

- **KISS:** *"È questa l'architettura più semplice possibile che soddisfa i requisiti attuali, funzionali e non funzionali?"* → sì
- **YAGNI:** *"Stiamo costruendo solo ciò che serve ora per vendere prodotti alla nostra base utenti iniziale?"* → sì. **Non aggiungiamo componenti o livelli per ipotetiche esigenze di scala future per milioni di utenti, perché non ne abbiamo ancora bisogno.**

La definizione di "buona architettura" (concetto chiave della lezione)

Una buona architettura non si misura dal numero di scatole nel diagramma né dalla sua complessità. Una buona architettura si misura sulla base dell'*adeguatezza allo scopo* (fitness for purpose).

Le due domande di verifica:

1. Il design soddisfa **efficacemente ed efficientemente** i requisiti dichiarati?
2. Opera **entro i vincoli definiti**?

Se la risposta è sì a entrambe, il design è buono — anche se è "solo una scatola".

Stack tecnologico del corso

- Linguaggio: **C#**
- Piattaforma: **.NET / ASP.NET Core**
- Tooling: **Visual Studio, .NET Aspire**

Nota importante della lezione: i principi architetturali e i design pattern **non dipendono dalla tecnologia**. Gli stessi design sono implementabili con **Java + Spring Boot, Python + Django/Flask, Node.js + Express**.

Code: E-Shop Monolithic App w/ KISS & YAGNI

Passo **CODE** del ciclo.

Il metodo delle lezioni di codice (4 passi, ricorrente per tutto il corso)

1. **Esaminare il codice** — struttura della soluzione, progetti chiave, file importanti, eventuali breakpoint per capire il flusso
2. **Eseguire l'applicazione** — con .NET Aspire, esplorando la dashboard, il grafico dei progetti e le variabili d'ambiente
3. **Testare i casi d'uso end-to-end** — navigare prodotti, aggiungere al carrello, ordinare
4. **Monitorare il comportamento** — console log, log strutturati, tracce, log del database

L'obiettivo **non** è scrivere codice riga per riga da zero, ma **validare il design**.

Definizione: **.NET Aspire** — un framework dell'ecosistema .NET per **lo sviluppo di applicazioni distribuite**. Serve a definire e configurare le diverse parti dell'applicazione e come si connettono, e fornisce una **dashboard** per monitorarle. In questa prima iterazione fa pochissimo, ma diventerà centrale più avanti.

Struttura della soluzione (cartella **1-esshop-monolith-one-app**)

eshop-monolith/

```
|— AppHost/          ← progetto .NET Aspire (orchestrazione)
|  |— Program.cs     ← qui si dichiara: "esiste 1 solo progetto: WebApp"
|— ServiceDefaults/
|  |— WebApp/        ← IL MONOLITE
|     |— Components/
|         |— Pages/   ← UI (pagine Blazor: Products, Cart, Orders)
|         |— Layout/
|— Data/           ← Entity Framework: DbContext, entità
|— Models/         ← DTO ed entità di dominio
|— Program.cs      ← configurazione app
```

L'essenza dell'architettura monolitica, visibile nella struttura: rendering dell'UI, regole di business per prodotti e ordini, accesso ai dati, modelli di dominio — **tutto dentro il progetto WebApp.**

Cosa si osserva eseguendo

- La dashboard Aspire mostra **una sola risorsa: webapp**
- Il **resource graph** è banale: un singolo nodo, nessuna dipendenza, *"perché tutto è contenuto in un unico server applicativo"*
- I casi d'uso funzionano: lista prodotti → aggiungi al carrello → modifica quantità → checkout (nome, cognome, email, indirizzo) → l'ordine compare in "Orders" → il carrello si svuota
- Nei log si vedono console log, log strutturati (es. Serilog), tracce e metriche HTTP. In questa fase il **distributed tracing è poco interessante** perché non c'è nulla di distribuito — ma l'infrastruttura è già pronta per quando servirà.

Eval

"Se questa applicazione è sufficiente per i nostri clienti, allora è la versione finale. Tuttavia, con la crescita della base clienti, ci troveremo presto ad affrontare diverse sfide."

E infatti parte il problema successivo.

Problem: Customer Basket Items Disappeared When App Restarts

Secondo giro del ciclo. Passo **PROBLEM**.

Lo scenario simulato

1. L'utente aggiunge 2 articoli al carrello

2. Mentre sta rivedendo il carrello, il server ha un problema temporaneo — **crash, nuovo deployment, o manutenzione ordinaria**
3. Nella dashboard Aspire si clicca "**Stop resource**" per simularlo: la webapp si ferma, il browser tenta di riconnettersi
4. Si riavvia la risorsa
5. La connessione torna, **ma il carrello è vuoto. Tutti i dati sono spariti.**

La causa tecnica

Aprendo `WebApp/Program.cs` e cercando la riga `AddDbContext`, si scopre che il `DbContext` è configurato con un **database in-memory**.

Definizione: database in-memory — un database che vive interamente nella RAM del processo applicativo. Non scrive nulla su disco.

Conseguenza: **tutti i dati dell'applicazione — prodotti, ordini e soprattutto gli articoli del carrello — sono memorizzati nella memoria del processo.** Quando il processo termina (crash o riavvio), quella memoria viene **azzerata**. Al riavvio l'applicazione parte con un database vuoto.

L'impatto di business

La lezione insiste su questo passaggio, ed è tipico del ragionamento architetturale:

Livello	Conseguenza
Tecnico	Perdita di stato al riavvio del processo
Utente	Frustrazione: perde il carrello accuratamente selezionato
Business	Pessima UX → perdita di vendite e di fatturato

La soluzione da progettare

Archiviare i dati su **un supporto di archiviazione stabile e permanente, in grado di sopravvivere ai riavvii dell'applicazione: un database persistente.**

Design: E-Shop Monolithic App with a Persistent Database

Passo **DESIGN** del secondo giro.

Toolkit aggiornato

- **Requisiti funzionali:** invariati
- **Requisiti non funzionali:** disponibilità e performance restano, ma si aggiunge **il requisito cruciale che sottolinea tutti gli altri:**

Persistenza e durabilità dei dati — tutti i dati principali (catalogo prodotti, account utente, carrello, articoli, ordini) devono **sopravvivere al riavvio dell'applicazione** ed essere archiviati in modo affidabile.

- **Architetture apprese:** resta l'architettura monolitica
- **Nuovo pattern nel toolkit: persistenza dei dati su storage esterno**, che quasi sempre implica l'uso di un **sistema di database dedicato**

Il nuovo design

Client → [Applicazione monolitica] → [Database relazionale]

(UI + Business + Data Access — invariato)	(persistente, esterno, su server separato)
--	---

Cosa NON è cambiato: la natura monolitica del codice applicativo. L'app contiene ancora UI, logica di business e accesso ai dati tutto insieme.

Cosa È cambiato — la differenza cruciale: il database **non è più un componente in-memory legato al ciclo di vita del server applicativo**. È uno **storage separato e permanente**.

Perché risolve il problema: se la webapp si blocca o viene ridistribuita, i dati nel database **restano al sicuro e disponibili** quando l'applicazione torna online. Il ciclo di vita dei dati è ora **disaccoppiato** dal ciclo di vita del processo.

Scelta tecnologica: perché un database relazionale

Per un e-commerce tipico con **dati strutturati**, un database relazionale è *"un punto di partenza solido e tradizionale"*. Le opzioni: **PostgreSQL, MySQL, SQL Server, Oracle**.

Le due ragioni date:

1. **Gestiscono bene le relazioni tra i dati** (Order → OrderItems → Product; Basket → BasketItems)
2. **Garantiscono forte coerenza (ACID)**, fondamentale per l'elaborazione degli ordini

Il corso sceglie **PostgreSQL**. La scelta *quale* database — relazionale vs NoSQL — sarà approfondita nella **Lezione 10**.

Code: E-Shop Monolithic App with a PostgreSQL Database

Passo **CODE** del secondo giro. Cartella 2-eshop-monolith-add-db.

Le due modifiche chiave al codice

1. In `AppHost/Program.cs` (il progetto `Aspire`)

```
builder.AddPostgres(...) ← aggiunge un SERVER PostgreSQL all'ambiente distribuito
    .AddDatabase("db") ← aggiunge un DATABASE sotto quel server
    .WithPgAdmin() ← aggiunge il container pgAdmin per esplorare il DB

builder.AddProject<WebApp>()
    .WithReference(db) ← inietta la CONNECTION STRING nella webapp
    .WaitFor(db) ← la webapp attende che il DB sia pronto
```

Cosa fa `WithReference`: Aspire **inietta automaticamente la connection string** come variabile d'ambiente nella webapp. Non c'è configurazione manuale di host/porta/credenziali.

2. In WebApp/Program.cs

```
AddNpgsqlDbContext<...>("db") ← al posto del database in-memory
```

Npgsql è il provider .NET per PostgreSQL, configurato per usare la connection string iniettata.

Nella cartella Data/Migrations ci sono le **migrazioni di Entity Framework**, che creano lo schema del database all'avvio. *(Il corso non approfondisce le migrazioni EF: è un corso di architettura, non di ORM.)*

Cosa cambia all'esecuzione

Prerequisito nuovo: Docker Desktop deve essere in esecuzione, perché Aspire scarica ed esegue il **container PostgreSQL da Docker Hub**.

La dashboard ora mostra **4 risorse** invece di 1:

- postgres (il server)
- db (il database dentro il server)
- pgadmin (il container per esplorare il DB)
- webapp (il monolite)

Nel resource graph si vede la **dipendenza**: webapp → db.

La verifica end-to-end (il momento chiave)

1. Si aggiungono 2 articoli al carrello
2. Si fa **Stop** sulla webapp → nella dashboard si nota che **il container PostgreSQL resta in esecuzione**: solo l'applicazione si è fermata
3. Si fa **Start** sulla webapp
4. Si torna al carrello → **gli articoli sono ancora lì** ✓

Verifica diretta sul database con pgAdmin: si espandono Servers → Database → Schemas → public → Tables. Si vedono le tabelle products, orders, baskets, basket_items.

Eseguendo SELECT su baskets si vede 1 carrello; su basket_items si vedono **2 righe**. Prova inconfutabile che i dati sono realmente persistiti.

Si completa poi il checkout e si verifica in pgAdmin che la riga sia comparsa nella tabella orders.

Nei log si vede: **all'avvio le migrazioni creano lo schema**; nelle tracce si vede la webapp che si connette a PostgreSQL per ogni operazione (lista prodotti, aggiunta al carrello, ecc.).

Problem: Tightly Coupled UI Limits Flexibility and User Experience

Terzo giro del ciclo. Passo **PROBLEM**.

Stato attuale

Una singola webapp contenente:

- **logica di presentazione** — pagine e componenti Blazor che renderizzano l'UI
- **logica di business** — gestione prodotti, ordini, carrello

- **codice di accesso ai dati**

Tutto integrato e distribuito insieme, connesso a PostgreSQL.

Il problema

Un'interfaccia utente strettamente accoppiata (tightly coupled) al backend limita la flessibilità e l'esperienza utente. Quando l'UI è strettamente accoppiata alla logica di backend, **svilupparle in modo indipendente diventa estremamente difficile.**

Le 6 sfide specifiche

1. Colli di bottiglia nello sviluppo

Se sviluppatori front-end e back-end **si interferiscono costantemente a vicenda nella stessa codebase**, lo sviluppo rallenta (conflitti di merge, build condivise).

2. Lock-in tecnologico sull'UI

Attualmente si usa **Blazor Server**, strettamente integrato e vincolato al backend. *E se volessimo usare **React, Vue o Angular** per un'UI più ricca e dinamica, o una vera SPA? È molto difficile farlo in modo pulito quando l'UI è profondamente integrata.*

3. Mancanza di flessibilità nello scaling

L'UI che serve **risorse statiche** ha profili di carico completamente diversi dalla logica che **elabora ordini**. Con un monolite unico, puoi solo scalare tutto insieme.

4. Impossibilità di sviluppo multicanale ← *la più critica per il business*

Il design attuale è **web-centrico**. Ma:

- *E se volessimo lanciare un'app mobile nativa per iOS e Android?*
- *E se un partner commerciale volesse integrarsi per elencare i nostri prodotti sulla sua piattaforma?*

Questi client non possono consumare pagine HTML. Hanno bisogno di un'API. E un'UI strettamente accoppiata non espone naturalmente un'API.

5. Innovazione UI/UX lenta

Non si può iterare rapidamente su UX e design visivo, perché **ogni modifica all'UI richiede un ciclo completo di build, test e deploy del backend.**

6. Specializzazione dei team compromessa

I front-end vogliono i loro strumenti e workflow; i back-end si concentrano sul lato server. Un'integrazione eccessiva **confonde i ruoli** e rende la specializzazione meno efficiente.

Impatto

Impedisce di offrire una migliore customer experience e di **espandere la presenza su tutti i canali**. La soluzione: **architettura a tre livelli**.

Learn: 3-Tier Architecture — Separating Presentation, Application, and Data

Passo **LEARN** del terzo giro.

Definizione

L'architettura a tre livelli (3-tier) è un modello client-server in cui l'applicazione è suddivisa in tre livelli FISICI distinti.

I tre tier

1. Presentation Tier (livello di presentazione)

- È quello con cui **l'utente interagisce direttamente**
- Responsabile di **visualizzare le informazioni** e **inviare i comandi dell'utente** al livello applicativo
- Esempi: browser che esegue una **SPA**, app mobile nativa, applicazione desktop
- **Dove gira:** sul computer client, o su web server che servono i file statici

2. Application Tier (livello applicativo / middle tier / logic tier)

- È **"il cervello" del sistema**
- Contiene la **logica di business core**, elabora i dati, prende decisioni, gestisce la comunicazione tra presentation e data tier
- **Espone le API** consumate dal presentation tier
- **Dove gira:** sull'application server

3. Data Tier (livello dati / database tier)

- Responsabile di **archiviazione e recupero dei dati**
- Costituito dal DBMS (PostgreSQL, SQL Server, ...) e dai dati stessi
- Garantisce **persistenza e integrità** dei dati
- **Dove gira:** sui database server

Il collante: presentation e application tier comunicano tramite **chiamate API su HTTP/HTTPS**; application e data tier comunicano tramite il protocollo del database.

I 6 motivi per usare la 3-tier

Motivo	Spiegazione
Separation of concerns	Ogni livello gestisce le proprie responsabilità specifiche → sistema più facile da comprendere e gestire
Scalabilità indipendente ★	<i>"Se il sito riceve molte visite ma la logica di business non è intensiva, puoi aggiungere web server al presentation tier senza scalare application tier o database."</i>
Flessibilità tecnologica	React/Angular per la presentazione, .NET/Java/Python per l'applicazione, PostgreSQL per i dati — la tecnologia migliore per ciascun compito
Manutenibilità	I team si specializzano e lavorano in parallelo. Definiti i contratti API , è meno probabile che una modifica a un livello rompa un altro
Sicurezza	Misure di sicurezza diverse per ciascun livello (es. il database non è raggiungibile da internet)
Riusabilità ★	Le API dell'application tier possono essere consumate da qualsiasi

Motivo

Spiegazione

client: SPA web, app mobile nativa, desktop, sistemi di terze parti

Come risolve il problema specifico

1. Il **presentation tier diventa un'applicazione completamente separata**: una **SPA** (React, Angular, Vue.js) che gira nel browser dell'utente
2. Il monolite si evolve fino a diventare principalmente **application tier**, esponendo **API REST o GraphQL** — non più pagine HTML
3. Le API di backend, ora **chiaramente definite**, non solo servono la SPA ma sono **già pronte a supportare app mobile native o altri client senza modifiche significative al backend**

"Questa è la chiave per una strategia multicanale."

★ Layers vs Tiers — la distinzione più importante della lezione

Questo punto è **quello che si confonde più spesso**:

	Layer (strato)	Tier (livello)
Natura	Logica	Fisica
Cos'è	Un modo di organizzare il codice sorgente in componenti con responsabilità diverse	Una separazione fisica di deployment
Dove vivono	Tutti gli strati possono coesistere in una singola applicazione distribuita	Ogni livello gira su un ambiente/server separato
Esempio	Presentation layer, Business logic layer, Data access layer — tutti dentro lo stesso .jar / stessa webapp	Browser (T1) + Application server (T2) + Database server (T3)

Il punto cruciale: il monolite delle sezioni precedenti **era già "layered"** (aveva Components/, Data/, Models/) ma **non era "tiered"** — era tutto in un unico processo di deploy.

E la relazione tra i due: **all'interno di un singolo tier puoi comunque avere più layer**.

L'application tier della nuova architettura conterrà internamente un API layer, un business logic layer e un data access layer.

Architettura interna vs esterna

Corollario importante che anticipa il resto del corso:

- **Architettura esterna** = come i deployment unit sono organizzati tra loro (monolite / 3-tier / microservizi)
- **Architettura interna** = come è organizzato il codice **dentro** ciascuna unità

*"Come architettura esterna possiamo seguire l'architettura a microservizi. Ma **ogni microservizio può seguire architetture interne diverse**: N-layer, Vertical Slice Architecture, Clean Architecture."*

Design: E-Shop Monolithic App with 3-Tier Architecture

Passo **DESIGN** del terzo giro.

Toolkit aggiornato

Requisiti funzionali: invariati.

Nuovi requisiti non funzionali:

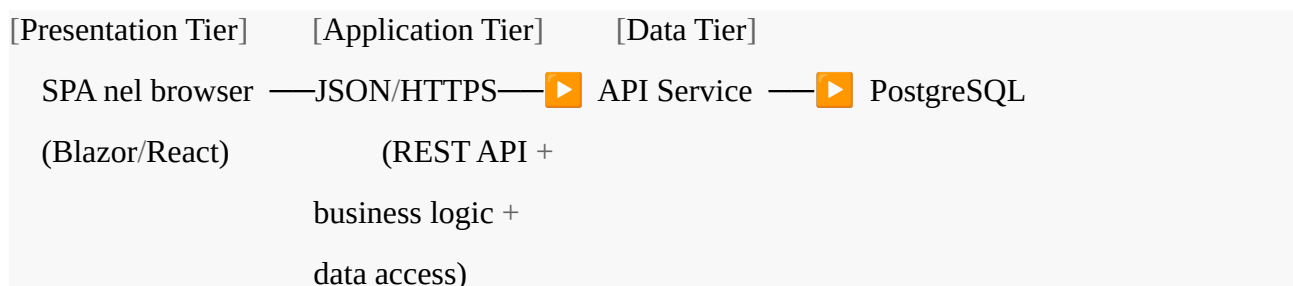
1. **Presentation tier separato** — l'UI deve poter essere sviluppata e distribuita **indipendentemente dal backend**
2. **Backend API-driven** — l'application tier deve esporre **API ben definite** consumabili da qualsiasi client
3. **Client multipli** — in futuro app mobile e altri client
4. **Scalabilità cross-tier** — poter scalare **indipendentemente** SPA, API server e database

Nuovi vincoli/assunzioni: servono **nuove competenze** — sviluppo SPA e **API design**, che diventa cruciale.

Nuove voci nel toolkit:

- Pattern: **architettura 3-tier**
- Principi: **separation of concerns**, **best practice di API design**

Il design risultante



Presentation tier: SPA che gira nel browser. Gestisce tutta l'UI, la presentazione e l'interazione. Comunica col backend **esclusivamente tramite chiamate API**, scambiando **JSON su HTTPS**.

Application tier: è il monolite di prima, che si evolve. Mantiene internamente business logic layer e data access layer, ma **espone API RESTful** e **non serve più pagine HTML direttamente all'utente finale**. Riceve le richieste dalla SPA, applica la logica di business, interagisce col database.

Data tier: PostgreSQL, invariato.

Scelte tecnologiche per tier

Tier	Scelta del corso	Alternative
Presentation	Blazor (come SPA/WebAssembly)	React, Vue, Angular
Application	ASP.NET Core Web API	Java/Spring, Python, Node.js
Data	PostgreSQL	altro RDBMS

Il punto cruciale: la 3-tier **consente la selezione indipendente della tecnologia per ogni livello**, sfruttando lo strumento più adatto a ciascuna funzione. Questo era **impossibile** nel monolite.

Sezione 11 — "Code: E-Shop with 3-Tier Architecture (SPA, Backend API & Database)"

Passo **CODE** del terzo giro. Cartella 3-eshop-monolith-add-ui.

La struttura Aspire: due categorie di risorse

In `AppHost/Program.cs` compare una distinzione concettuale importante:

Categoria	Cosa rappresenta	Nel nostro caso
Backing services	Database, message broker, cache e altri sistemi esterni/distribuiti a cui i progetti si connettono	PostgreSQL, pgAdmin
Projects	I nostri servizi — backend, frontend, ecc.	ApiService, WebApp

Codice (semplificato):

```
// DATA TIER

var db = builder.AddPostgres("postgres")
    .WithDataVolume() ← i dati sopravvivono anche allo stop del container
    .WithLifetime(...)
    .WithPgAdmin()
    .AddDatabase("db");

// APPLICATION TIER

var apiService = builder.AddProject<ApiService>()
    .WithReference(db).WaitFor(db);

// PRESENTATION TIER

builder.AddProject<WebApp>()
    .WithReference(apiService).WaitFor(apiService);
```

Nota `WithDataVolume()`: aggiunge un **volume Docker** così che i dati del database persistano anche quando il container viene fermato e riavviato — *"utile per uno sviluppo più realistico"*.

Il grafico delle dipendenze è ora una **catena**: `WebApp` → `ApiService` → `db`. È **letteralmente il diagramma 3-tier, reso eseguibile**.

Lato presentation: come la WebApp chiama l'API

In `WebApp/Program.cs` si registra un **HttpClient**:

```
builder.Services.AddHttpClient<ApiServiceClient>(client =>
{
    client.BaseAddress = new Uri("https+http://apiservice");
});
```

Concetto chiave: Service Discovery. Il nome `apiservice` non è un hostname reale — corrisponde al nome della risorsa dichiarata in `AppHost`. **Il service discovery di .NET Aspire risolve automaticamente `apiservice` all'URL interno corretto** dove gira l'API. Questo evita di hardcodare URL e porte.

In `WebApp/ApiClient/ApiServiceClient.cs` ci sono metodi come:

- `GetProducts()` → esegue una **HTTP GET** verso l'endpoint `/products` dell'`ApiService`
- `GetProductById(id)`
- `GetBasket()`
- ecc.

La conseguenza architetturale è esattamente ciò che volevamo: *"il progetto WebApp non accede più direttamente al database né contiene la logica di business per il recupero dei prodotti. Si limita a chiamare l'API Service."*

Lato application: l'ApiService

In `ApiService/Program.cs` si registra la connessione PostgreSQL (è l'`ApiService`, ora, a parlare col DB).

In `ApiService/Endpoints/` si trovano gli endpoint esposti con **Minimal API**:

```
app.MapGet("/products", async (AppDbContext db) =>
    await db.Products.ToListAsync());
```

In `ApiService/Models/` ci sono i modelli di dominio su cui opera la logica di business.

Risultato: l'API Service **incapsula la logica di business e l'accesso ai dati**, fornendo dati a **qualsiasi client capace di parlare HTTP** — la SPA di oggi, l'app mobile di domani, il partner commerciale dopodomani.

La verifica con il distributed tracing ★

Questa è la parte più istruttiva. Nella dashboard **Aspire**, sezione **Traces**, aprendo una traccia si vede la **catena completa** di una singola richiesta utente:

```
Browser → WebApp    (caricamento pagina prodotti)
WebApp → ApiService  (HTTP GET /products)
ApiService → PostgreSQL (SELECT * FROM products)
← risposta risale la catena
```

Ed è questa la vera differenza rispetto alle iterazioni precedenti: **ora c'è un flusso distribuito da tracciare**. Nel monolite era tutto in-process e la traccia era inutile.

Si completa poi il caso d'uso: aggiungi al carrello → checkout (nome, cognome, email, indirizzo) → il carrello si svuota e compare l'ordine. Nei log si vedono le **richieste HTTP DELETE/POST** dalla WebApp all'ApiService, e i comandi **INSERT** dall'ApiService al database.

Big Ball of Mud

Breve sezione concettuale, ma centrale per capire perché si passerà oltre.

Definizione

Un "Big Ball of Mud" è un sistema software con una struttura caotica e mal organizzata, in cui il codice è così aggrovigliato da risultare difficile da comprendere, mantenere o estendere.

Caratteristiche:

- Nessuna struttura o organizzazione chiara
- Tutto ammassato alla rinfusa: **difficile distinguere dove finisce una parte e ne inizia un'altra**
- Nel tempo si accumulano soluzioni rapide e scorciatoie → **accumulo di debito tecnico**

Le 3 cause

1. **Cicli di sviluppo troppo rapidi** — nella fretta di rilasciare funzionalità, le best practice vengono ignorate, producendo una codebase disordinata e piena di scorciatoie
2. **Mancanza di progettazione adeguata** — senza design e pianificazione, il sistema cresce in modo disordinato; le decisioni prese in fretta portano a **incongruenze**
3. **Mancato refactoring** — il codice va ripulito e ottimizzato regolarmente; se non lo fai, la gestione diventa via via impossibile

Nota concettuale importante: il Big Ball of Mud **non è sinonimo di monolite**. È un *anti-pattern strutturale*. Esistono monoliti ben strutturati (i **modular monolith** della Lezione 3) e — peggio — esistono microservizi che formano un "distributed Big Ball of Mud", ovvero il fango con in più la latenza di rete.

Problem: Growing Pains — Our Monolith is Becoming Hard to Manage and Evolve

Questo è l'**EVAL** del terzo giro, che diventa il **PROBLEM** della Lezione 3.

L'osservazione chiave

"Sebbene l'API Service sia ben disaccoppiato dal presentation tier grazie all'interfaccia API, il codice sorgente interno dell'API Service rimane essenzialmente monolitico."

Tutta la logica di business — **prodotti, ordini, pagamenti, account utente** — risiede dentro una singola unità di deployment.

Abbiamo risolto l'accoppiamento esterno (UI ↔ backend), non quello interno.

Lo scenario di crescita

L'applicazione ha **successo**. Il business cresce rapidamente:

- si vogliono aggiungere velocemente molte funzionalità per restare competitivi: **programma fedeltà, ricerca avanzata, raccomandazioni personalizzate, integrazione con più metodi di pagamento**
- il **team di sviluppo** cresce e si formano sotto-team per area: **team Catalogo prodotti, team Ordini, team Pagamenti**

"Questa crescita è positiva per il business, ma esercita un'enorme pressione sull'API backend interna, indifferenziata e monolitica."

Le 6 sfide che emergono

1. Complessità del codice

Funzionalità di aree di business diverse si accumulano nella stessa codebase → **groviglio disordinato**. Diventa difficile navigare, capire il flusso dei dati, valutare l'impatto di una modifica — **anche per i membri esistenti del team**.

2. Onboarding difficile

Comprendere l'intero sistema diventa un peso per i nuovi sviluppatori. La **curva di apprendimento è ripida e frustrante**.

3. Mancanza di confini chiari

"Le modifiche apportate da un team a una singola funzionalità possono inavvertitamente rompere una parte completamente diversa dell'applicazione."

Questo è il problema centrale: **nessun confine strutturale impedisce al team Pagamenti di rompere il Catalogo**.

4. Velocità di sviluppo ridotta

Più team che lavorano sulla stessa codebase ampia producono:

- **frequenti conflitti di merge**
- **tempi di build e test più lunghi**
- **costi di coordinamento significativi**

5. Lavoro parallelo impossibile

I team non riescono a lavorare sulle rispettive funzionalità **in modo veramente indipendente** né a rilasciarle senza essere bloccati dagli altri.

6. Rischio crescente delle modifiche

Più la codebase è complessa, più cresce **la paura di modificarla**. Anche una piccola modifica può avere conseguenze diffuse e impreviste → **ogni deploy diventa un evento ad alto rischio**.

La conclusione della lezione

*"L'architettura monolitica ci ha permesso di **iniziare rapidamente**, ma ora è diventata un **collo di bottiglia**. Impedisce alla nostra azienda in crescita e ai nostri team in espansione di rilasciare nuove funzionalità in modo efficiente e sicuro."*

Il requisito per la prossima iterazione:

*"Abbiamo bisogno di un modo per organizzare la codebase monolitica in **moduli interni ben definiti e a basso accoppiamento**, che possano essere sviluppati e compresi in modo più indipendente, **anche quando vengono distribuiti insieme**."*

→ Soluzione: **Modular Monolith Architecture** (Lezione 3).

Nota il dettaglio: *"anche quando vengono distribuiti insieme"*. Non si sta ancora saltando ai microservizi. Si vuole prima il **beneficio della modularità senza il costo del sistema distribuito**.